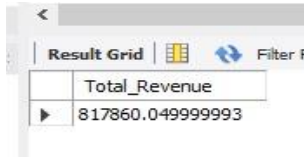


PIZZA SALES SQL QUERIES

A. KPI's

1. Total Revenue:

```
SELECT SUM(total_price) AS Total_Revenue FROM pizza_sales;
```

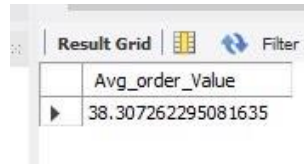


A screenshot of a SQL query result grid. The grid has two columns: 'Total_Revenue' and a value '817860.049999993'. The interface includes a 'Result Grid' tab, a 'Filter' button, and a 'Refresh' button.

Total_Revenue
817860.049999993

2. Average Order Value:

```
SELECT (SUM(total_price) / COUNT(DISTINCT order_id)) AS Avg_order_Value  
FROM pizza_sales;
```



A screenshot of a SQL query result grid. The grid has two columns: 'Avg_order_Value' and a value '38.307262295081635'. The interface includes a 'Result Grid' tab, a 'Filter' button, and a 'Refresh' button.

Avg_order_Value
38.307262295081635

3. Total Pizzas Sold:

```
SELECT SUM(quantity) AS Total_pizza_sold FROM pizza_sales;
```

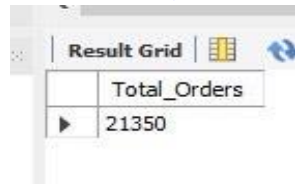


A screenshot of a SQL query result grid. The grid has two columns: 'Total_pizza_sold' and a value '49574'. The interface includes a 'Result Grid' tab, a 'Filter' button, and a 'Refresh' button.

Total_pizza_sold
49574

4. Total Orders:

```
SELECT COUNT(DISTINCT order_id) AS Total_Orders FROM pizza_sales;
```

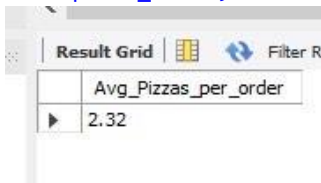


A screenshot of a SQL query result grid. The grid has two columns: 'Total_Orders' and a value '21350'. The interface includes a 'Result Grid' tab, a 'Filter' button, and a 'Refresh' button.

Total_Orders
21350

5. Average Pizzas Per Order:

```
SELECT CAST(CAST(SUM(quantity) AS DECIMAL(10,2)) /  
CAST(COUNT(DISTINCT order_id) AS DECIMAL(10,2)) AS DECIMAL(10,2))  
AS Avg_Pizzas_per_order  
FROM pizza_sales;
```

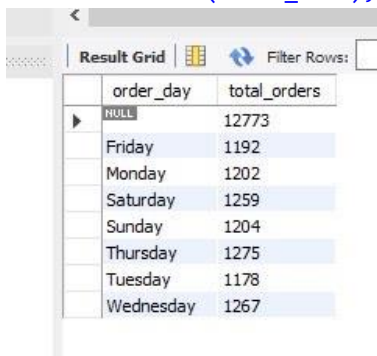


A screenshot of a SQL query result grid. The grid has two columns: 'Avg_Pizzas_per_order' and a value '2.32'. The interface includes a 'Result Grid' tab, a 'Filter' button, and a 'Refresh' button.

Avg_Pizzas_per_order
2.32

B. Daily Trend For Total Orders

```
SELECT DAYNAME(order_date) AS order_day,  
COUNT(DISTINCT order_id) AS total_orders  
FROM pizza_sales  
GROUP BY DAYNAME(order_date);
```

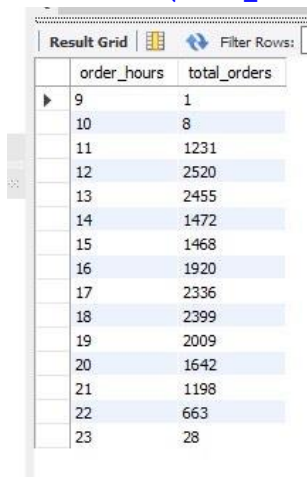


The screenshot shows a database query result grid with two columns: 'order_day' and 'total_orders'. The data is grouped by day of the week. The 'order_day' column lists the days from Friday to Wednesday, with a 'NULL' entry at the top. The 'total_orders' column shows the corresponding counts for each day.

order_day	total_orders
NULL	12773
Friday	1192
Monday	1202
Saturday	1259
Sunday	1204
Thursday	1275
Tuesday	1178
Wednesday	1267

C. Hourly Trend For Orders

```
SELECT HOUR(order_time) AS order_hours,  
COUNT(DISTINCT order_id) AS total_orders  
FROM pizza_sales  
GROUP BY HOUR(order_time)  
ORDER BY HOUR(order_time);
```



The screenshot shows a database query result grid with two columns: 'order_hours' and 'total_orders'. The data is grouped by hour of the day, ranging from 9 to 23. The 'total_orders' column shows the corresponding counts for each hour.

order_hours	total_orders
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28

D. Percentage of Sales By Pizza Category

```
SELECT pizza_category, CAST(SUM(total_price) AS DECIMAL(10,2)) as  
total_revenue,  
CAST(SUM(total_price) * 100 / (SELECT SUM(total_price) from pizza_sales)  
AS DECIMAL(10,2)) AS PCT  
FROM pizza_sales  
GROUP BY pizza_category;
```

Result Grid			
Filter Rows:			
	pizza_category	total_revenue	PCT
▶	Classic	220053.10	26.91
	Veggie	193690.45	23.68
	Supreme	208197.00	25.46
	Chicken	195919.50	23.96

E. Percentage of Sales By Pizza Size

```
SELECT pizza_size, CAST(SUM(total_price) AS DECIMAL(10,2)) as
total_revenue,
CAST(SUM(total_price) * 100 / (SELECT SUM(total_price) from pizza_sales)
AS DECIMAL(10,2)) AS PCT
FROM pizza_sales
GROUP BY pizza_size
ORDER BY pizza_size;
```

Result Grid			
Filter Rows:			
	pizza_size	total_revenue	PCT
▶	L	375318.70	45.89
	M	249382.25	30.49
	S	178076.50	21.77
	XL	14076.00	1.72
	XXL	1006.60	0.12

F. Total Pizzas Sold By Pizza Category

```
SELECT pizza_category, SUM(quantity) as Total_Quantity_Sold
FROM pizza_sales
WHERE MONTH(order_date) = 2
GROUP BY pizza_category
ORDER BY Total_Quantity_Sold DESC;
```

Result Grid		
Filter Rows:		
	pizza_category	Total_Quantity_Sold
▶	Classic	492
	Chicken	406
	Supreme	394
	Veggie	364

G. Top Five Best Sellers By Total Pizzas Sold

```
SELECT pizza_name,
SUM(quantity) AS Total_Pizza_Sold
FROM pizza_sales
GROUP BY pizza_name
ORDER BY Total_Pizza_Sold DESC
LIMIT 5;
```

Result Grid			Filter Rows:
	pizza_name	Total_Pizza_Sold	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	

H. Bottom Five Best Sellers By Total Pizzas Sold

```
SELECT pizza_name,
SUM(quantity) AS Total_Pizza_Sold
FROM pizza_sales
GROUP BY pizza_name
ORDER BY Total_Pizza_Sold ASC
LIMIT 5;
```

Result Grid			Filter Rows:
	pizza_name	Total_Pizza_Sold	
▶	The Brie Carre Pizza	490	
	The Mediterranean Pizza	934	
	The Calabrese Pizza	937	
	The Spinach Supreme Pizza	950	
	The Soppressata Pizza	961	

NOTE:

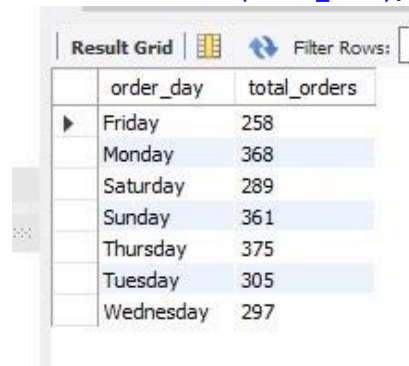
If you want to apply the Month, Quarter, Week filters to the above queries you can use WHERE clause. Follow some of the examples below.

```
SELECT DAYNAME(order_date) AS order_day,
COUNT(DISTINCT order_id) AS total_orders
FROM pizza_sales
WHERE MONTH(order_date) = 1
GROUP BY DAYNAME(order_date);
```

Result Grid			Filter Rows:
	order_day	total_orders	
▶	Friday	148	
	Monday	136	
	Saturday	133	
	Sunday	122	
	Thursday	143	
	Tuesday	72	
	Wednesday	66	

*HERE MONTH(order_date) = 1 indicates that the output is for the month of January. MONTH(order_date) = 4 indicates output for Month of April.

```
SELECT DAYNAME(order_date) AS order_day,  
COUNT(DISTINCT order_id) AS total_orders  
FROM pizza_sales  
WHERE QUARTER(order_date) = 1  
GROUP BY DAYNAME(order_date);
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains two columns: 'order_day' and 'total_orders'. The data is as follows:

order_day	total_orders
Friday	258
Monday	368
Saturday	289
Sunday	361
Thursday	375
Tuesday	305
Wednesday	297

*Here QUARTER(order_date) = 1 indicates that the output is for the Quarter 1. Quarter(Order_date) = 3 indicates output for Quarter 3.